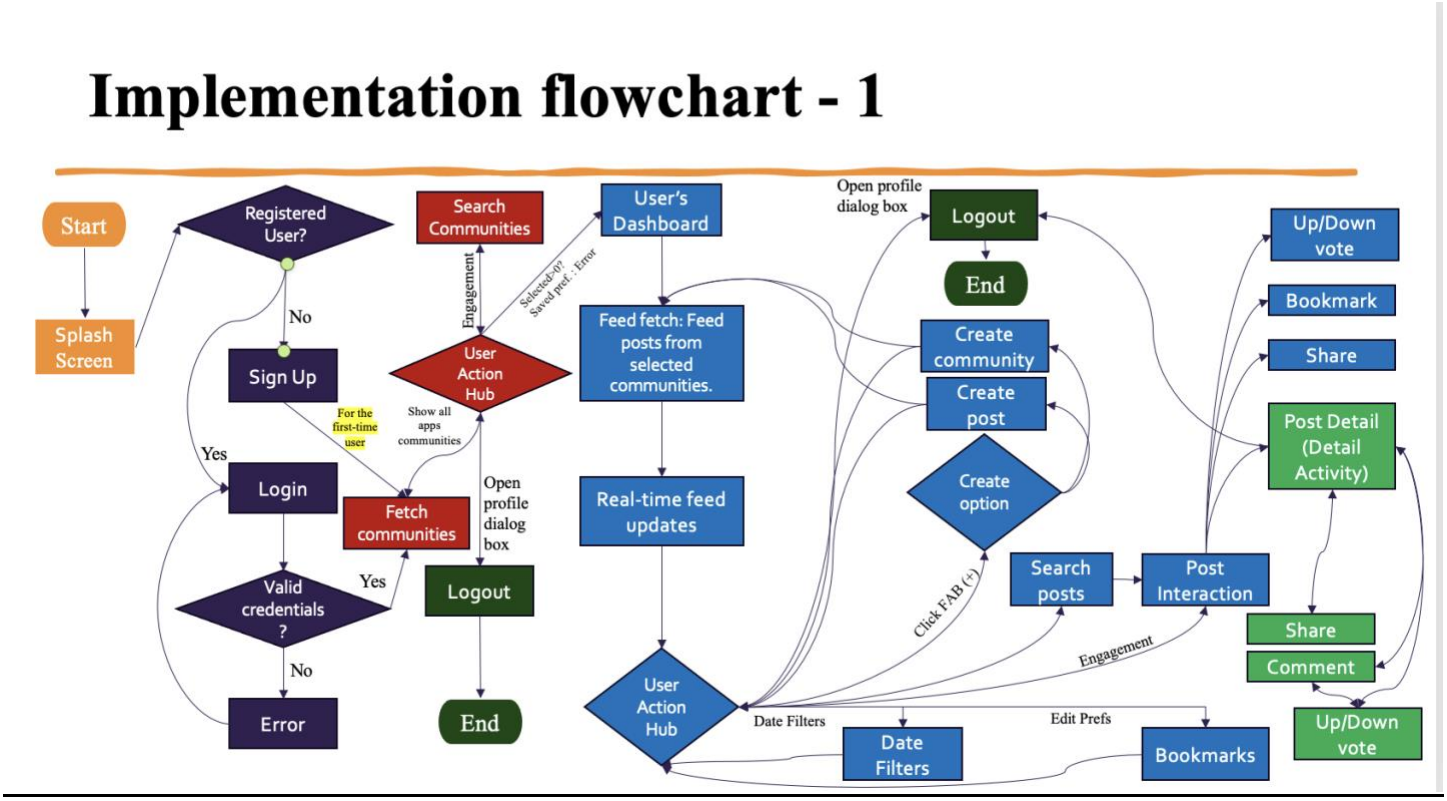
HARSH PATEL, Final project, ITMD 555, May 6th, 2026

App name: Nexus

GitHub Link: https://github.com/harshjpatel/ITMD555Project_Nexus |
Professor (GitHub username: @igitiit) got added as a collaborator already.

Index	Topic	Page Number
1	Overview about Nexus, Personalization, app features, APIs used (Text)	2
2	Layouts in res/layout/ (with screenshots)	3-6
3	How this app works? (with screenshots)	6-10
4	All used API proofs and Extra credits (with screenshots)	11-12
5	Nexus app on the device	12
6	Overall major application workflow	13



Overview

- Nexus is a community platform, like Reddit, that bridges fragmented social discussions to deliver high-quality information and meaningful discourse.

- Users can join or start community pages, create posts on any community, and view posts from others.

- Communities include topics like: Business, Entertainment, Health, Science, Sports, Technology, Gaming, Traveling, MacBook, or any random interest-based groups.

Groups and Personalization

- Users begin by customizing their post feed through filtration (e.g., selecting/deselecting categories like Creative Design, Retro, etc.). This ensures the Nexus Dashboard is relevant from the very first login.

Current Features

- Like (i.e., upvote) / Dislike (i.e., downvote) posts and comments
- Thread-based commenting system
- Share posts with other users
- Bookmark posts using **"Recaps"** for quick access
- Search content directly from main feed
- Community-based feed with filtering options
- Create and engage across multiple communities
- Firebase-powered backend (NoSQL database)
- Can see recent post in App widget.
- **Cloud Messaging (FCM):** Used for push notifications (via MyFirebaseMessagingService).
- **AppWidget API**: Powers the home screen widget (AppWidgetProvider, RemoteViews).
- **WorkManager API**: Handles reliable background tasks like the WidgetUpdateWorker.
- **AlarmManager API**: Specifically used in your PostWidget to bypass the standard 30-minute widget update limit for high-frequency (60s) refreshes.

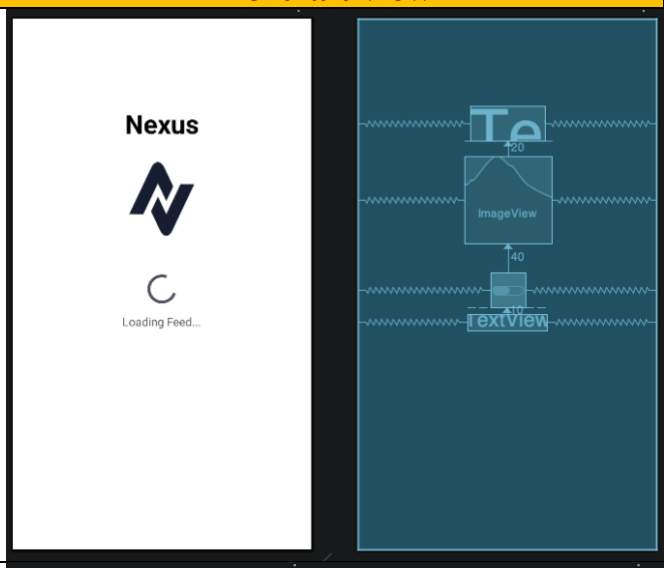
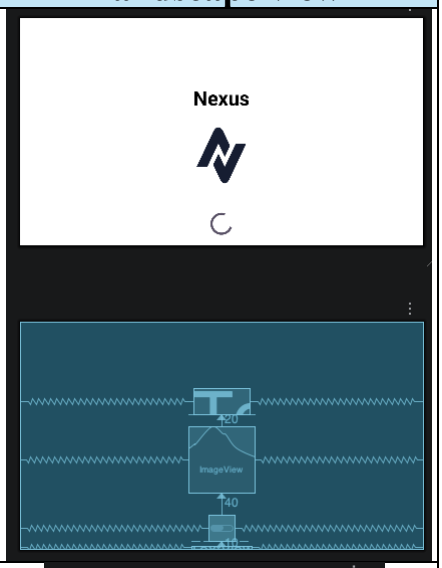
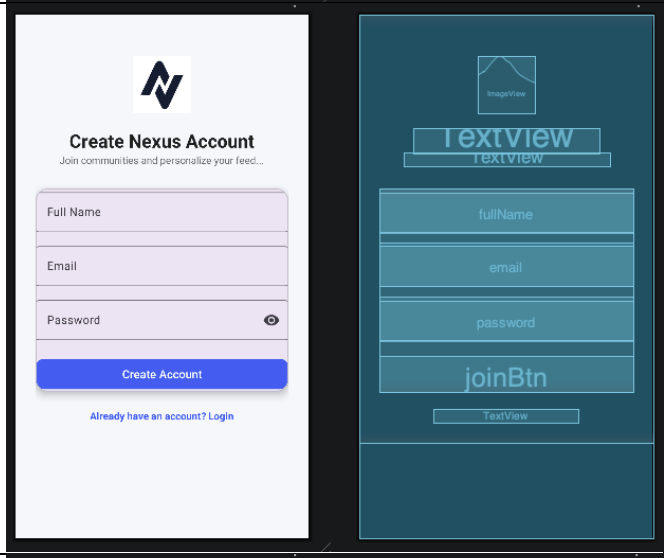
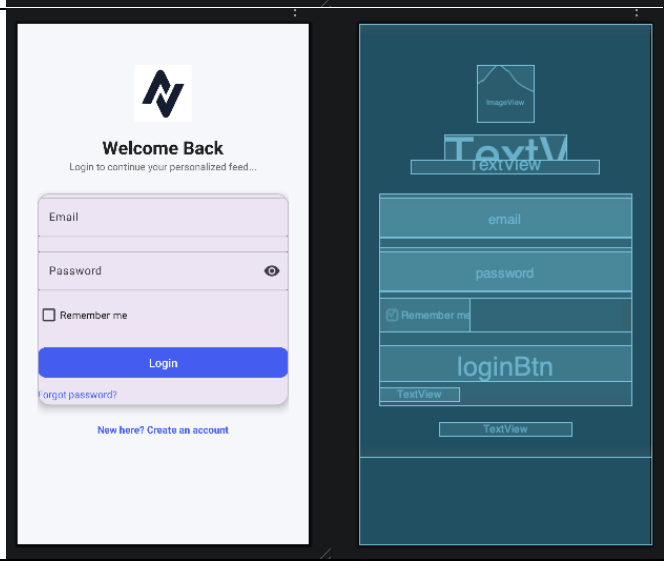
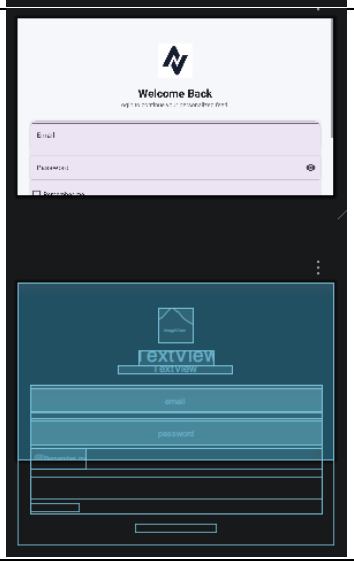
Backend and Firebase Services
(*Professor email added to collaborator: https://console.firebase.google.com/u/1/project/camscannerai/overview*)

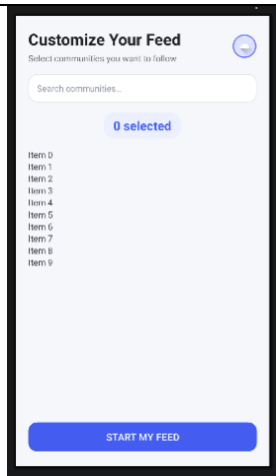
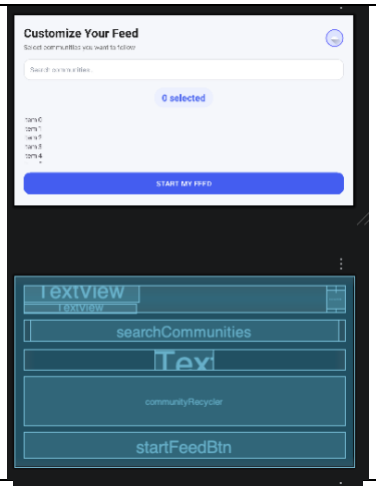
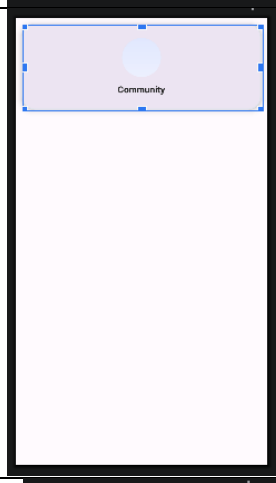
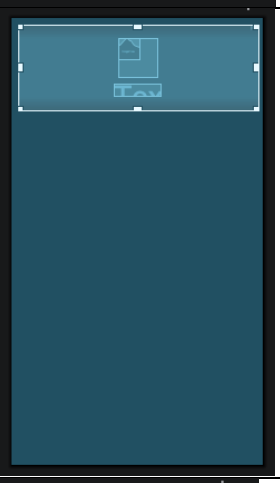
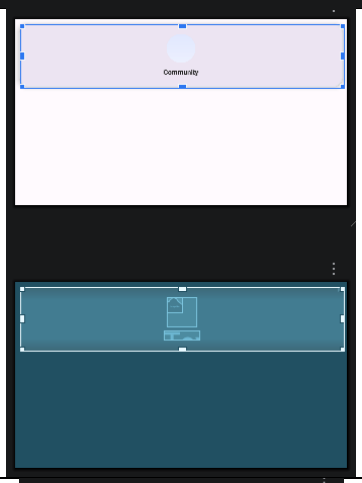
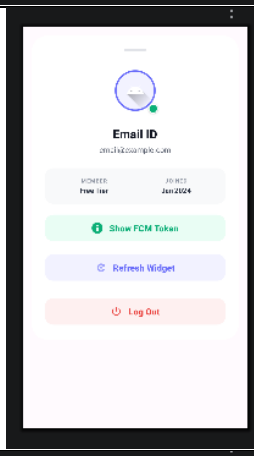
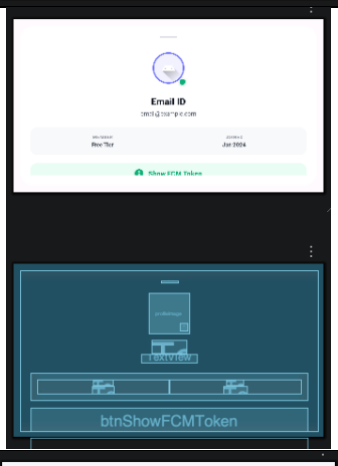
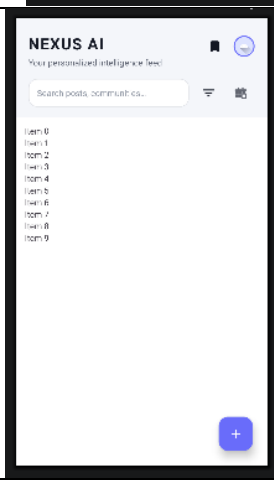
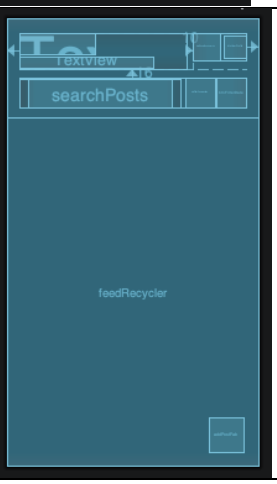
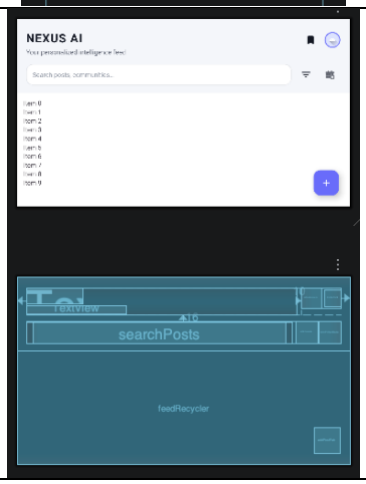
- Firebase Authentication (user login & identity management)
- Cloud Firestore (database for users, posts, likes, comments, etc.)

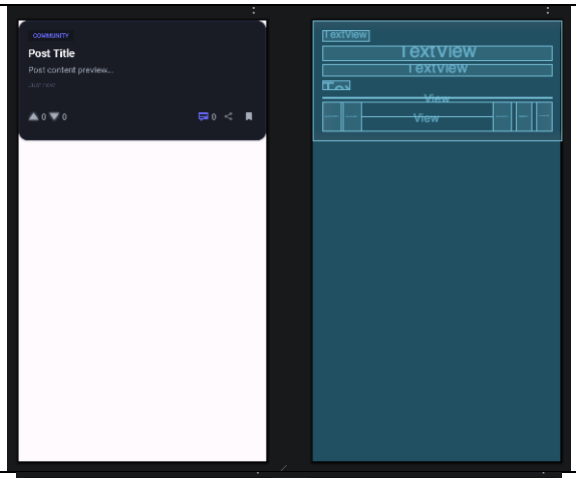
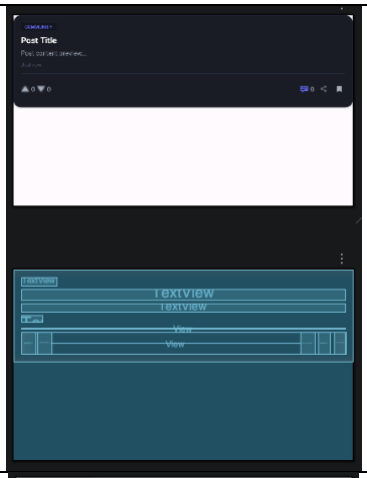
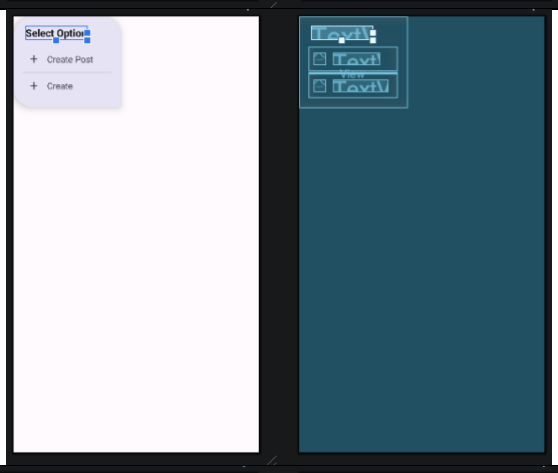
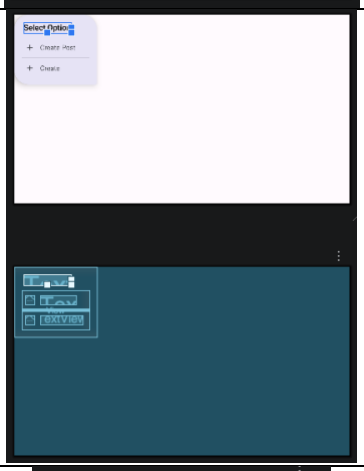
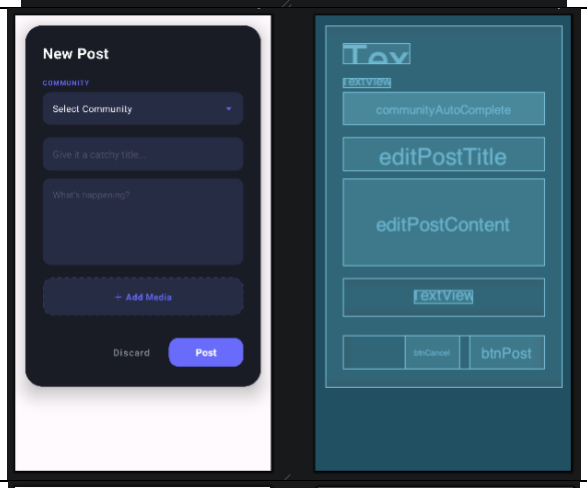
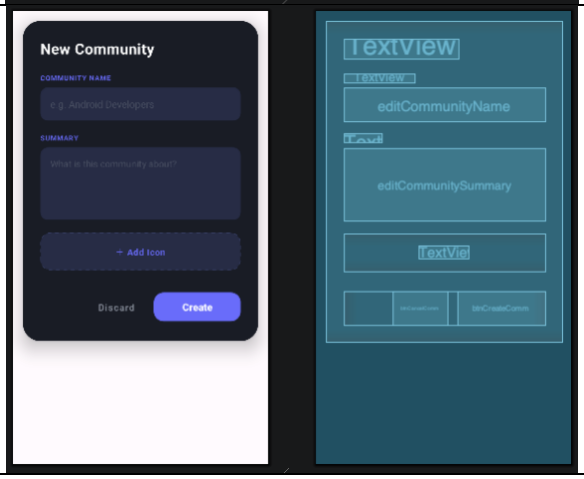
APIs used:

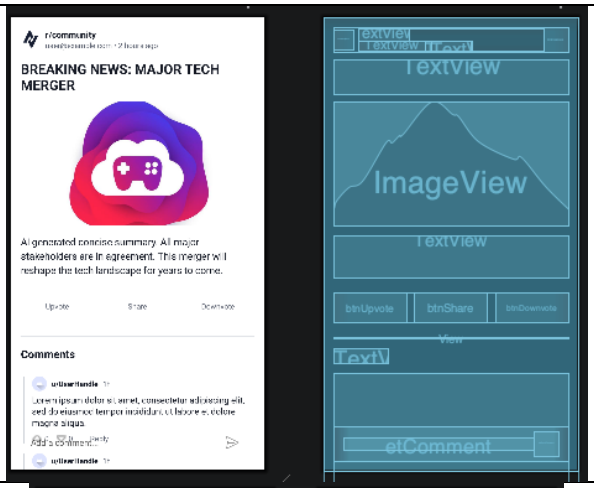
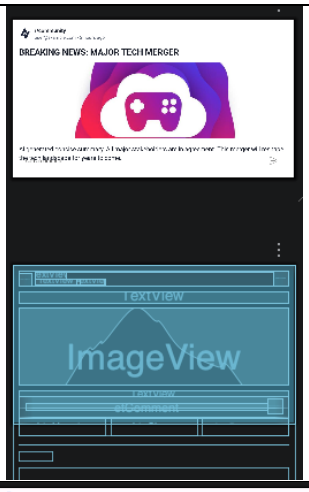
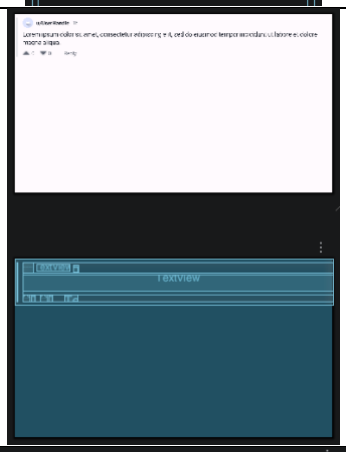
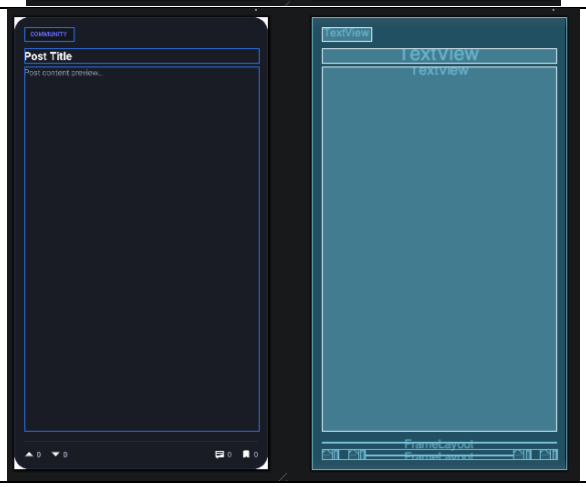
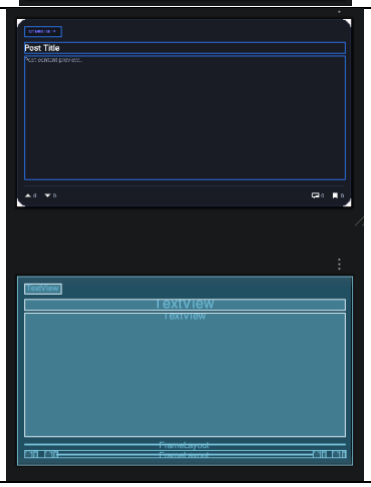
- Firebase Firestore API
- Firebase Auth API
- Firebase Cloud Messaging (FCM) API
- Firebase Analytics API
- Android AppWidget API
- Android WorkManager API
- Android AlarmManager API
- Glide API
- Google Material Design Components API
- Android Jetpack (AppCompat/ConstraintLayout) APIs
- Custom Share Preview API (Local Python Server)

Layouts in res/layout/:

	Portrait view	Landscape view
activity_main.xml		
activity_signup.xml		
activity_login.xml		

activity_customize_feed.xml				
community_item.xml				
dialog_user_profile.xml				
activity_dashboard.xml				

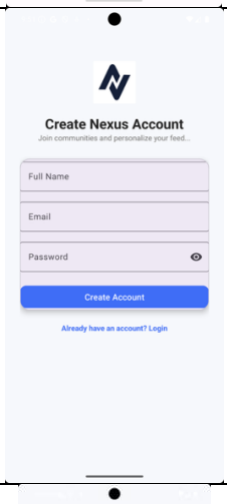
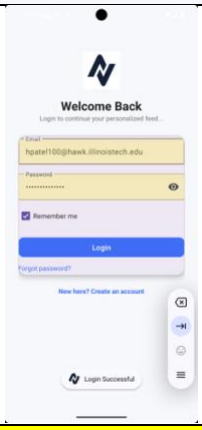
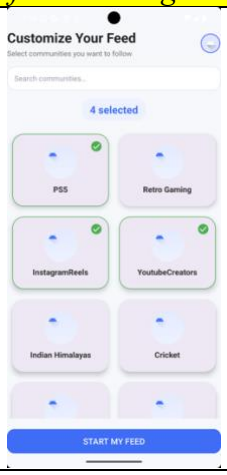
item_post.xml			
dialog_select_option.xml			
dialog_create_post.xml			
dialog_create_community.xml			

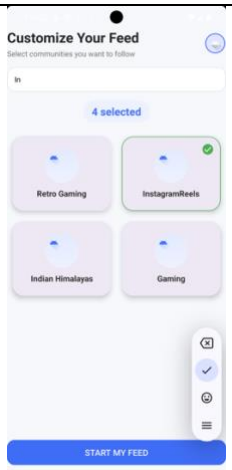
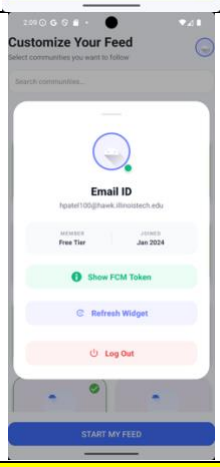
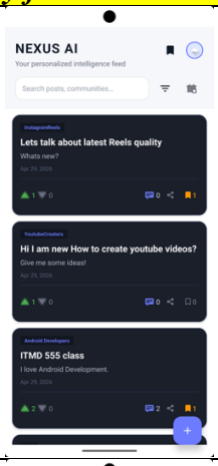
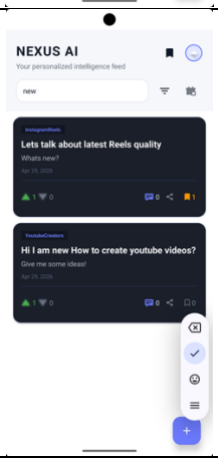
activity_post_detail.xml			
item_comment.xml			
post_widget.xml			

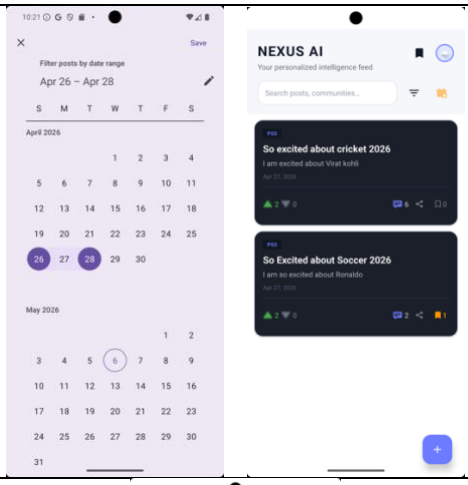
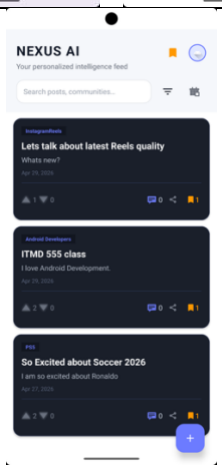
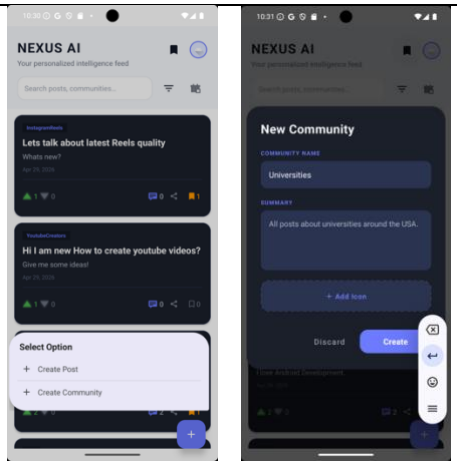
How this app works

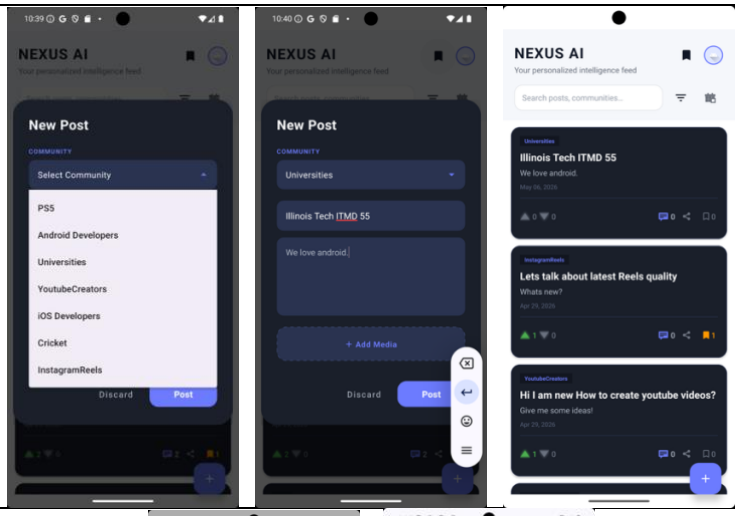
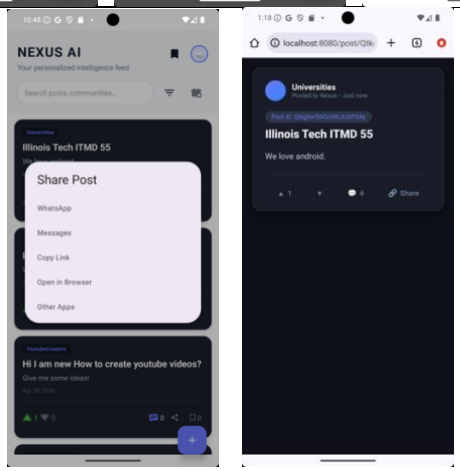
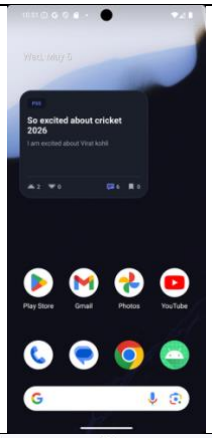
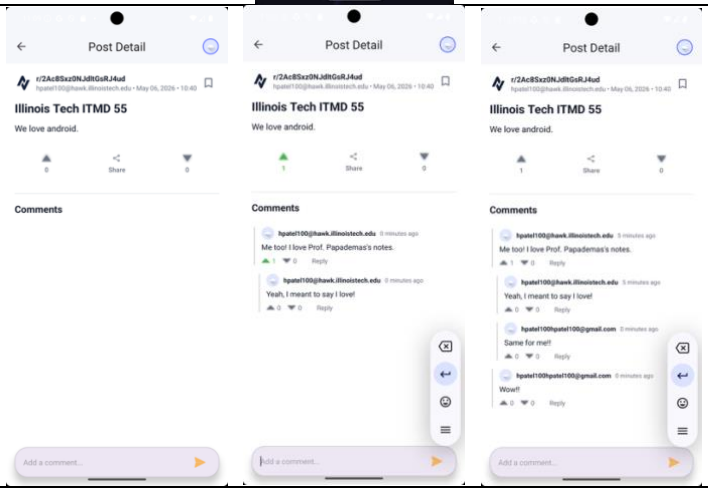
(Both landscape and portrait mode of login screen successful/unsuccessful attempt)

Things to see	Home Screen Portrait + Landscape view	Comments (if any)

App launcher screen *(When app starts)* *Splash screen*		Splash screen for few seconds.
Sign up Activity		Sign up if user installed the app first time. The date will be saved in Firebase user authentication.
Sign-in Activity		Sign in if user signed up before. It will match credential with Firebase user authentication.
Things to do: *After selecting "Login" OR First-time "Sign-up"*		
Customize Feed Activity Screen *(Showing how many communities selected by a particular user)*		User can saved their preference for selected comminuties. User can updaate anytime.

Customize Feed Activity Screen **<u>Feature-1</u>:** *(search communities by a specific keyword in search bar, with total overall communities selcted by a user)*		User can search communities by using searchbar.
Customize Feed Activity Screen **<u>Feature-2</u>:** *(User's profile with refresh widget and logout options)*		User's profile to generate FCM token, Refresh widget, and logout options.
***Things to do:** After selecting **"Start My feed"** button that saved a partivulr user's dashboard feed references.*		
Dashboard Activity Screen *(<u>User can see posts from selected communities</u>, latest post seen first, with scrollable U/I)* *(**<u>NOTE</u>**: User can upvote (green) OR downvote (red) post, bookmark or share the post)*		User's feed from selected communities before.
Dashboard Activity Screen **<u>Feature-1</u>:** *(<u>search posts by a specific keyword in search bar</u>)*		User can search posts by using searchbar.

Dashboard Activity Screen **<u>Feature-2</u>:** *(<u>search posts by choosing a date range</u> and then save option on top-right)*		User can search posts by using choosing specific date range.
Dashboard Activity Screen **<u>Feature-3</u>:** *(<u>Click the top-right bookmark button</u> [i.e., top-right bookmark's orange color showing bookmark view is ON] to see all the bookmarked post with search bar option)*		User can bookmark particulr posts to visit it again in the future by selecting bookmark button.
Dashboard Activity Screen **<u>Feature-4</u>:** *(<u>Create a new community page</u>, it will automatically store into Firebase DB)*		User can create a new post from selecting communities before.
Dashboard Activity Screen **<u>Feature-5</u>:** *(<u>Add/drop community settings with filter option button anytime</u> (i.e., right side of search bar) so you can see relevent posts on activity dashboard)* *Here we selected "universities" community again.*		User can again select the new ones OR deselect the selected comunities.

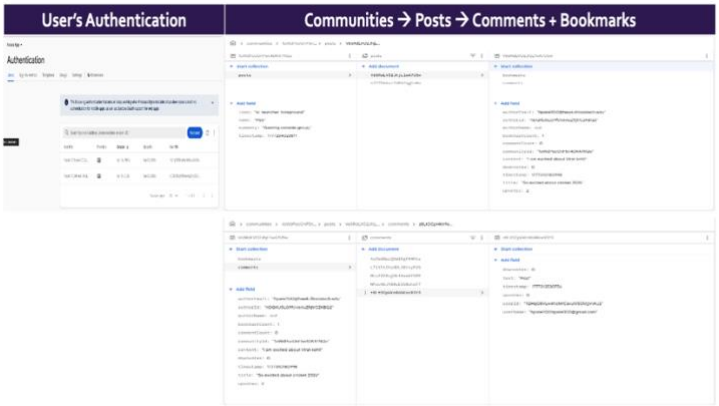
Dashboard Activity Screen **<u>Feature-6</u>:** *(<u>Create a new post from user's selected communities</u>, it will automatically store into Firebase DB;* *new post will be shown as top most post)*		User can create a new community.
Dashboard Activity Screen **<u>Feature-7</u>:** *(<u>User can share the post with others</u>, if I share it on browser, as I don't have any server registered for this app, I am using localhost)*		**<u>To see it, you need to</u>** **1. Start server program, and** **<u>On 1st terminal screen:</u>** *python3 server.py* **2. Allocate 8080 port to emulator** **<u>On 2nd terminal screen:</u>** *adb reverse tcp:8080 tcp:8080*
(Extra credit) **Widget Features showing most interacted post** **<u>Feature-1</u>:** *(<u>Widget features</u>, Users can see the most interacted post from activity dashboard in 'Nexus' app widget)* *Btw, You need to add Nexus widget in device.*		**<u>To force a widget refresh, use following command in terminal:</u>** *adb shell am broadcast -a com.example.nexus.UPDATE_WIDGET_MANUAL -n com.example.nexus/.PostWidget* **<u>Note</u>**: 1. You must be logged in. 2. You have selected communities. 3. And if it's a single post for selected communities.
Detailed post Activity Screen **<u>Feature-1</u>:** *(User can select a particular post, can see upvote/downvote, bookmark, and share option)* *You can see who made the post, his ID, email ID and timestamp of a particular post and dynamic update.* ***(NOTE:** User can see the comments, can reply to a particular comment or start a new comment thread.)*		**<u>Note</u>**: 1. You can see post from User1 *(hpatel100@hawk.illinoistech.edu)*. 2. User1's comments as well in comment threads. 3. Anuther USER-2 *(hpatel100hpatel100@gmail.com)*'s comment as well. 4. So, its dynamic and any user can upvote/downvote/reply or share, bookmark the posts. 5. Sharing the post is same like above dashbaord activity's feature-7. 6. Upvote/downvote/bookmmark is the same features like above dashbaord activity's feature-1 and feature-3. 7. The 1st and 2nd screnshot is from User-1. 8. 3rd screenshot is from User2's screen.

APIs used proofs and Extra Credits

API-1 (user login/signup) + API-2 (storing and retrieving data)

Firebase DB implementation

Code of google-services.json is in the peoject code file.



API-1: You can see User authentication db in Firebase.

API-2: You can also see collections and subcollections of each collections:

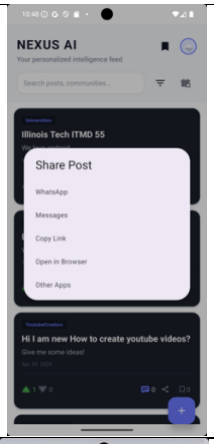
- Communities
  - Posts
    - Bookmarks
    - Comments

Proof:
DashboardActivity.class + PostDetailActivity.class's FirebaseFirestore.getInstance() method.

API-3 (Custom Python Web API)

local server.py backend

(i.e., people share a post on reddit and can see it "on broswer" without interacting it, so I created my own rest api to see it on browser so user can see it without logged it).



The Custom Backend API (server.py)
A Python script that starts a server on http://localhost:8080.
Endpoint: it listens for requests at /post/{post_id}.

Logic: It accepts query parameters (like title, content, and author) and dynamically generates a full HTML "Web View" of a post.

Language: Python (using http.server). In PostDetailActivity.java (and FeedAdapter.java) code.

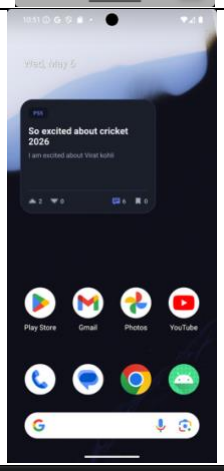
It's a REST API (A custom web API I created)
1. Endpoint: /post/
2. Protocol: HTTP (running on port 8080).
3. Communication: Your Android app "talks" to this server to generate shareable links.

Even you can copy localhost link and check on Macbook broswer.

API-4 + (Extra credit-1) Background/Widget Features showing most interacted post

Feature-1:
(Widget features, Users can see the most interacted post from activity dashboard in 'Nexus' app widget)

Btw, You need to add Nexus widget in device.



To see it before you share, you need to
1. **Start server program, and**

On 1st terminal screen:
python3 server.py

2. **Allocate 8080 port to emulator**

On 2nd terminal screen:
adb reverse tcp:8080 tcp:8080

Every time the widget updates, it sets an AlarmManager timer to trigger itself again in 60,000ms (1 minute). This creates a continuous loop where the widget pulls fresh data from Firestore every minute.

When a push notification arrives with a data payload, it calls updateWidget().This triggers the WidgetUpdateWorker, which forces the widget to refresh immediately without waiting for the 1-minute timer.

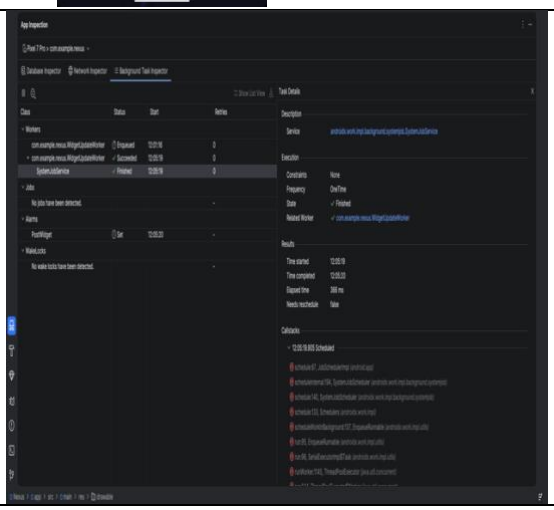
NOTE: However, in some Android devices limits this to a minimum of 30 minutes to save battery.

Note:
1. You must be logged in.
2. You have selected communities.
3. And if it's a single post for selected communities.

(Extra credit-2)

Service-1+2: Background Task Management

API Utilized: Android Jetpack WorkManager API (androidx.work) + AlarmManager



Real-time Monitoring: Using the Android Studio Background Task Inspector, execution was verified on a live device.

The inspector confirms that the WidgetUpdateWorker successfully transitioned from an "Enqueued" state to "Succeeded."

The background task was efficiently handled by the system's JobScheduler, completing the update cycle in 386ms.

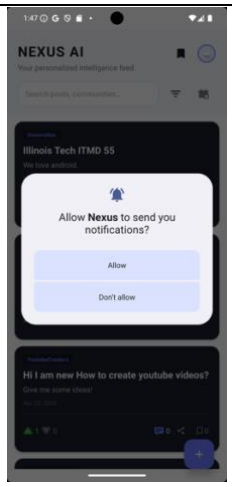
The screenshot provides a full audit trail, showing the exact time of execution (12:05:19) and the underlying system callstacks that managed the task.

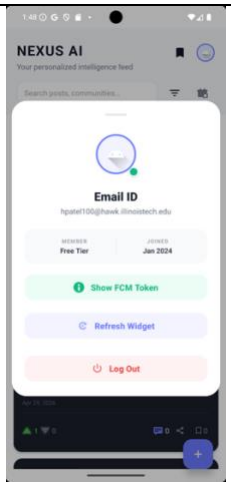
Proof:
WidgetUpdateWorker.java class and the triggerWidgetUpdate() method in DashboardActivity

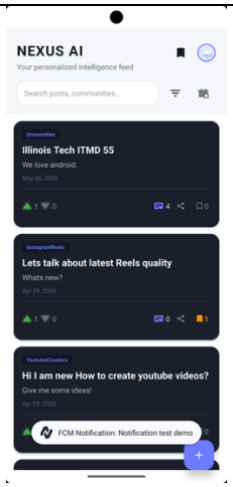
(Extra credit-3)

Service-3:
Firebase Cloud Messaging

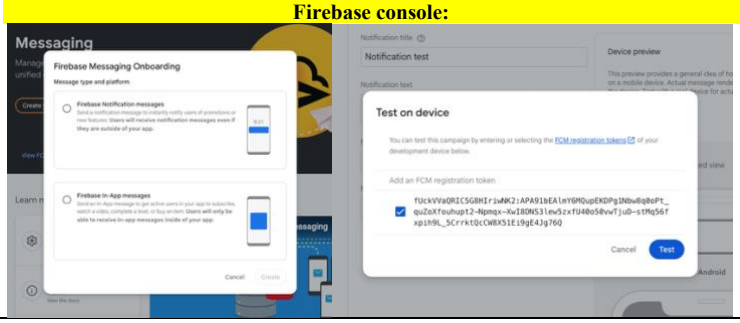
(Background services)







Firebase console:



MyFirebaseMessagingService: Created a custom service to handle incoming messages.

Foreground Handling: Implemented onMessageReceived to display a Toast notification when the app is open

FirebaseMessaging.getInstance().getToken() to fetch the unique device registration token.

Added a one-tap "Copy" feature to the token dialog, making it easy to grab the token for testing via the Firebase Console or curl.

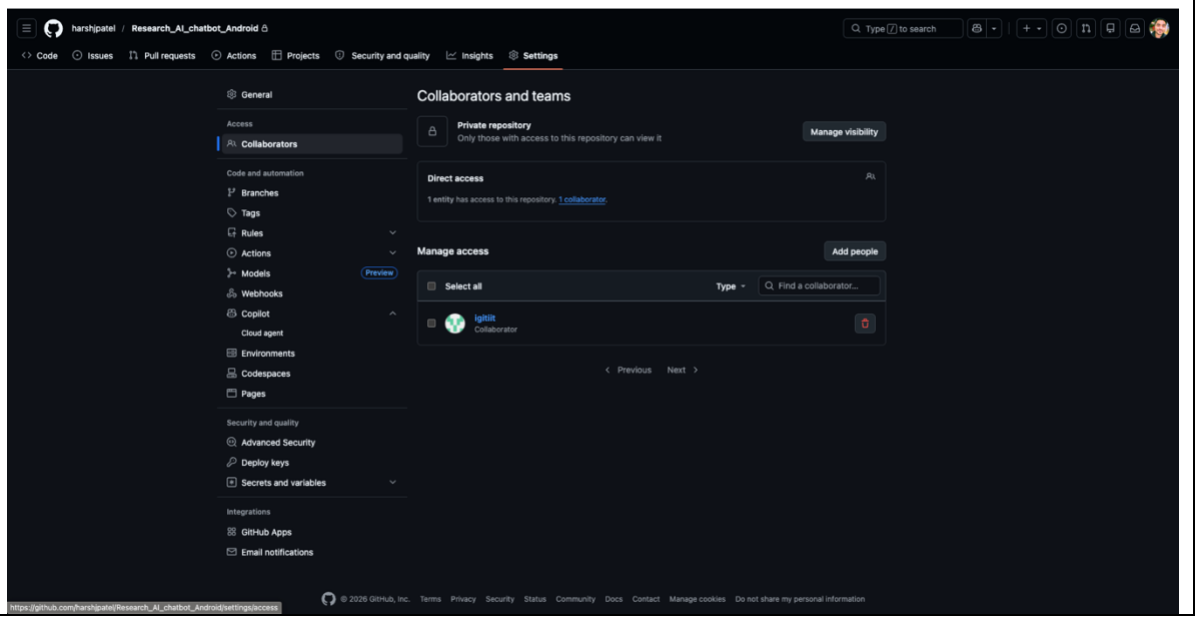
Verified that messages sent from the Firebase Console (V1 API) are successfully received by the device.

Visual confirmation as well on movile devie.

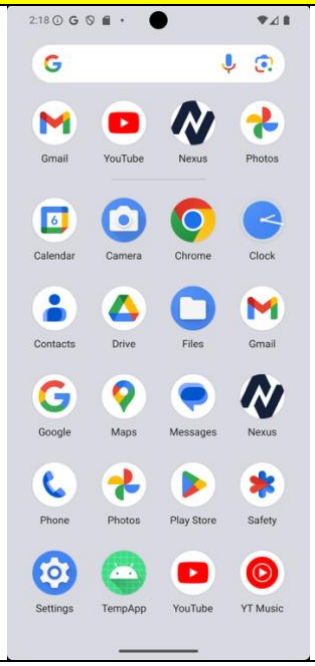
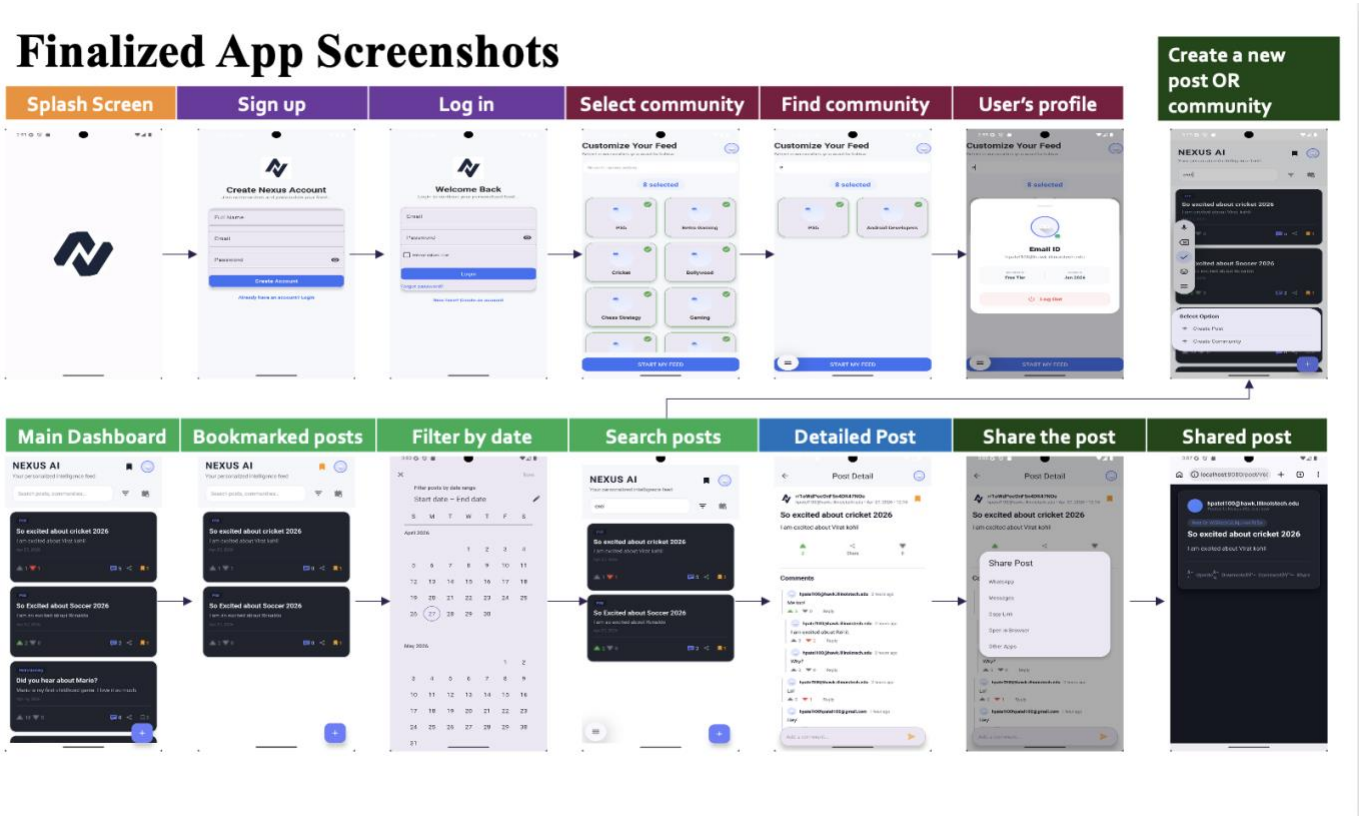
i.e., Copy token from show FCm Token from user's profile.

(Extra credit-4)
Github commits

GitHub Link: https://github.com/harshjpatel/ITMD555Project_Nexus | Professor (GitHub username: @igitiit) got added as a collaborator already.



(P.T.O.)

Nexus ***App in the app list (On the 3rd to the last bottom on the right side).***	**Nexus** launcher Screen
	

<u>Overall major application workflow: (I used .xml files only to create U/I)</u>